

The University of the West Indies

Department of Computing & Information Technology

**From Publication Archive to Research
Advisor: A Source-Grounded Research
Intelligence Platform for Academic
Discovery**

Design, Implementation, and Evidence-Constrained Evaluation
of the TTLAB Platform

A thesis submitted in partial fulfilment of the requirements for the degree of

MSc Data Science

Candidate: Jarod Esareesingh

Student ID: **AUTHOR INPUT REQUIRED: student ID**

Email: jarod.esareesingh@my.uwi.edu

Country: Trinidad and Tobago

Supervisor: **AUTHOR INPUT REQUIRED: supervisor name**

Submission: **AUTHOR INPUT REQUIRED: submission month and year**

Repository evidence snapshot: HEAD 713ea5c plus seven enumerated pre-existing working-tree changes, audited 11 July 2026.

Abstract

Academic publication lists make research visible but do not, by themselves, support full-text discovery, evidence-linked question answering, or the translation of prior work into feasible student projects. This thesis presents an evidence-based case study of the TTLAB Research Intelligence Platform, a local research-discovery prototype built with FastAPI, SQLite, React, and deterministic or locally hosted language-model components. The audited snapshot contains 134 publication records, 98 extracted PDFs, 698 pages, and 756 page-aware text chunks. It implements keyword, hashing-vector, and hybrid retrieval; citation-returning question answering; thesis-extension recommendations; topic and author exploration; generated paper artefacts; and an administrative review mechanism.

The contribution is a reproducible, source-traceable engineering artefact and a rigorous characterization of its current limits, rather than a claim of a novel retrieval algorithm or demonstrated user benefit. Repository inspection, database integrity checks, isolated automated tests, API probes, frontend compilation, and interface inspection establish implementation evidence. The active hashing-vector index covers only 25 of 756 chunks, 32.94% of chunks have an unknown inferred section, author identity defects remain, and no completed gold retrieval, question-answering, or human-review study exists. Consequently, persisted “grounded” statuses are interpreted only as structural and lexical citation checks, not factual accuracy. The thesis connects each research question to a method, evidence source, result, and bounded conclusion, and specifies the evaluation work required before effectiveness claims can be made.

Keywords: research intelligence, information retrieval, retrieval-augmented generation, scholarly recommendation, citation grounding, research software, reproducibility.

Declaration and Acknowledgements

AUTHOR INPUT REQUIRED: Insert the university-approved authorship declaration, disclosure of AI-assisted tools, copyright statement, and acknowledgements.

Contents

Abstract	i
Declaration and Acknowledgements	ii
List of Abbreviations	x
1 Introduction	1
1.1 Aim	1
1.2 Objectives	2
1.3 Contributions	2
1.4 Thesis structure	3
2 Problem Statement	4
2.1 Stakeholders and information needs	4
2.2 Scope	4
2.3 Success criteria	5
3 Research Questions and Objectives	6
3.1 Question-to-evidence design	6
3.2 Propositions not yet tested	6
4 Literature Review	8
4.1 Scholarly discovery and research intelligence	8
4.2 Scholarly recommendation and expertise discovery	8
4.3 Lexical and dense retrieval	9
4.4 Topic modelling and author-topic relations	9
4.5 Retrieval-augmented generation and grounding	9
4.6 Human review and responsible AI	10
4.7 Research gap	10
5 Methodology	11

5.1	Study design	11
5.2	Evidence protocol	11
5.3	Reproducible snapshot collection	12
5.4	Analytical boundaries	12
5.5	Proposed empirical methodology	12
6	System Architecture and Design	14
6.1	Architectural overview	14
6.2	Entity and provenance design	15
6.3	Traceability as a cross-cutting requirement	15
6.4	Operational architecture	16
6.5	Design decisions and alternatives	16
7	Data Collection and Processing	17
7.1	Corpus provenance	17
7.2	Ingestion workflow	17
7.3	Chunking	17
7.4	Corpus status	18
7.5	Metadata quality	19
7.6	Data minimization and retention	19
8	Implementation	20
8.1	Technology stack and interface	20
8.2	Keyword and hashing-vector retrieval	20
8.3	Hybrid scoring and diversification	23
8.4	Question answering	23
8.5	Thesis-extension recommendations	25
8.6	Topic, author, artefact, and review functions	25
9	Experimental Design	28
9.1	Completed technical verification	28
9.2	Implemented retrieval metrics	28
9.3	Proposed retrieval experiment	29
9.4	Proposed QA evaluation	29
9.5	Proposed recommendation evaluation	29
9.6	Proposed topic/author and summary evaluation	29
9.7	Evaluation workflow and current state	29
10	Results	32
10.1	Corpus and ingestion results (RQ1)	32

10.2 Retrieval and answer traceability (RQ2)	32
10.3 Recommendation traceability (RQ3)	33
10.4 Discovery, artefacts, and review (RQ4)	33
10.5 Reproducibility and verification (RQ5)	33
11 Discussion	35
11.1 From archive to inspectable advice	35
11.2 RQ1: coverage is useful but incomplete	35
11.3 RQ2: provenance survives, semantic coverage does not	35
11.4 RQ3: recommendation structure is safer than recommendation quality . . .	36
11.5 RQ4: reviewability is necessary but not sufficient	36
11.6 RQ5: reproducibility is stronger than evaluation maturity	36
11.7 Implications for research-intelligence systems	37
12 Limitations, Ethics and Threats to Validity	38
12.1 Technical limitations	38
12.2 Evaluation limitations	38
12.3 Ethical and responsible-AI considerations	39
12.4 Threats to internal validity	39
12.5 Threats to construct validity	39
12.6 Threats to external validity	40
12.7 Threats to reproducibility	40
13 Conclusion and Future Work	41
13.1 Prioritized future work	41
13.2 Final author decisions	42
A Research-Question Traceability Matrix	43
B Reproducibility Guide	45
B.1 Evidence snapshots	45
B.2 Application verification	45
B.3 Document builds	46
B.4 Snapshot identifiers	46
B.5 Screenshots	46
C Schema and API Summary	47
C.1 Principal persisted contracts	47
C.2 Principal API surfaces	48

D	Proposed Review Instruments	49
D.1	Answer and citation review	49
D.2	Recommendation review	49
D.3	Topic and author review	49
D.4	Minimum provenance fields	50

List of Figures

6.1	Implemented system architecture and evidence return path. Dashed flow emphasizes provenance and status returned to the interface.	14
6.2	Simplified entity and provenance model reconstructed from the SQLModel classes. JSON fields are summarized rather than expanded.	15
7.1	Implemented ingestion, extraction, indexing, and topic-processing pipeline with snapshot counts.	18
7.2	Paper-level PDF/extraction status in the audited database. Counts sum to the 134 seed records.	18
7.3	Distribution of publication years in the seed metadata. Values are descriptive of the audited snapshot, not a publication-rate analysis.	19
8.1	Dashboard in the audited browser session. The empty top-topic widget contrasts with populated explorer data and is retained as failure evidence. . . .	21
8.2	Search results with mode, scores, paper metadata, pages, and source snippets. The full-page capture is deliberately cropped to its top results for legibility. .	22
8.3	Implemented search and RAG workflow. The verifier checks structure, membership, and lexical overlap rather than semantic entailment.	23
8.4	Ask TTLAB response showing generated-answer notice, grounding state, and cited papers. The full-page capture is deliberately cropped to the answer and leading citations.	24
8.5	Implemented recommendation pipeline and the human-validation stage that has not yet been executed.	25
8.6	Extension Finder profile and leading ranked result with source/gap status. The full-page capture is deliberately cropped before additional result detail.	26
8.7	Discovery and review interfaces. The explorer screenshot retains the “Click to View” author artefact.	27
9.1	Implemented and proposed evaluation workflow. Gold retrieval/QA labels and human reviews are absent in the audited snapshot.	30

9.2 Evaluation dashboard correctly displaying not-run and placeholder states rather than inferred performance. 31

10.1 Keyword and hashing-index coverage. Database-marked hashing status overstates the 25-record active index. 32

List of Tables

2.1	Stakeholders, primary tasks, and principal failure risks.	5
3.1	Research-question traceability plan.	7
10.1	Current evaluation results and what they do not establish.	34
A.1	Connection from each research question to method, evidence, observed re- sult, and conclusion.	43
B.1	Key SHA-256 snapshot identifiers.	46
C.1	Principal persisted entities and traceability fields.	47
C.2	User-facing API groups and implemented purpose.	48

List of Abbreviations

API	Application Programming Interface
BM25	Best Matching 25 probabilistic retrieval function
DOI	Digital Object Identifier
FTS	Full-Text Search
LLM	Large Language Model
MRR	Mean Reciprocal Rank
MVP	Minimum Viable Product
PDF	Portable Document Format
QA	Question Answering
RAG	Retrieval-Augmented Generation
RQ	Research Question
TTLAB	AUTHOR INPUT REQUIRED: official expansion of TTLAB
UI	User Interface

Introduction

Research groups commonly expose their work through publication lists. Such lists answer what has been published, but leave substantial interpretation work to the reader: locating a paper, obtaining its full text, understanding its contribution, relating it to other work, and deciding whether it can support a thesis or collaboration. The precursor TTLAB system automated collection, display, summarisation, notifications, and podcast preparation over publication metadata and abstracts, while identifying retrieval-augmented querying as future work [1]. The present project extends that direction into a full-text research-intelligence prototype.

The broader scholarly-discovery landscape includes autonomous digital libraries such as CiteSeer [2], researcher-centred services such as ArnetMiner [3], large literature graphs such as S2ORC [4], and open bibliographic infrastructure such as OpenAlex [5]. These systems demonstrate the value of linked metadata and retrieval at scale. This project addresses a smaller and different setting: a curated laboratory corpus, local operation, explicit page/chunk evidence, student-facing extension advice, and review before public exposure.

The implemented application is not treated as proof that all of those goals have been achieved. Instead, this thesis adopts a strict evidential boundary. A capability is called implemented only when supported by code, persisted state, an executed test or probe, or a captured interface. A quality claim requires appropriate evaluation evidence. Proposed experiments and missing institutional decisions are marked explicitly.

1.1 Aim

The aim is to design, implement, and reproducibly characterize a source-traceable research-intelligence prototype for TTLAB publications, while identifying the evidence still required to establish retrieval effectiveness, answer faithfulness, recommendation usefulness, and readiness for public deployment.

1.2 Objectives

The project objectives are to:

- O1. construct a deterministic pipeline from publication records and local PDFs to page-aware, searchable chunks;
- O2. provide searchable publication, topic, and author views over a curated corpus;
- O3. implement retrieval and citation-returning question answering with visible evidence and grounding warnings;
- O4. produce source-linked thesis-extension recommendations that distinguish paper evidence from system suggestions;
- O5. expose generated outputs and metadata to administrative review rather than silently presenting them as verified; and
- O6. define and support a reproducible evaluation method that separates implementation checks from empirical effectiveness.

1.3 Contributions

The defensible contributions of the audited snapshot are:

- a modular local software artefact joining PDF extraction, page-aware chunking, hybrid retrieval, source-linked generation, exploration, and review;
- a structured recommendation contract separating baseline work, evidence status, inferred or explicit gaps, generated extension scope, risks, skills, and evaluation suggestions;
- an auditable evidence package containing machine-readable repository and runtime snapshots, reproducible figure sources, build commands, and an author-review checklist; and
- a candid characterization of coverage, data quality, index consistency, and evaluation gaps in a real laboratory corpus.

No algorithmic novelty, retrieval superiority, factual-accuracy rate, recommendation benefit, or deployed production service is claimed.

1.4 Thesis structure

Chapter 2 defines the problem and scope. Chapter 3 presents the research questions and traceability plan. Chapter 4 reviews relevant scholarship. Chapters 5 to 8 describe the evidence method, design, corpus, and implementation. Chapters 9 and 10 separate planned evaluation from currently observed results. Chapters 11 to 13 interpret the evidence, state threats and ethical requirements, and conclude.

Problem Statement

The practical problem is a mismatch between a publication archive and the questions its users need answered. A student may ask which paper is feasible to extend in one semester; an industry user may ask which researchers work on a problem; a researcher may need a public explanation of a technical contribution. Answering these questions requires evidence distributed across full papers, metadata, authorship relations, and generated interpretations.

Three risks make a naive chatbot unsuitable. First, metadata-only retrieval can omit methods, limitations, experiments, and future work that appear only in the paper body. Second, generated prose can merge retrieved facts with unsupported suggestions. Third, an apparently polished interface can obscure missing PDFs, incomplete indexes, parsing errors, or the absence of human validation. Retrieval-augmented generation (RAG) can make retrieved evidence available to a generator [6], but retrieval and citation display do not automatically make an answer correct.

The research problem is therefore:

How can a small, locally operated system transform a laboratory publication snapshot into a reproducible, inspectable research-advice workflow in which full-text evidence, generated interpretation, review state, and evaluation status remain distinguishable?

2.1 Stakeholders and information needs

2.2 Scope

The implemented scope is deliberately local and bounded: a manually inspectable corpus; SQLite persistence; local PDFs; deterministic processing; a React interface; an offline extractive answerer; and an optional local Ollama adapter. Authentication, production deploy-

Table 2.1: Stakeholders, primary tasks, and principal failure risks.

Stakeholder	Task	Principal risk
Student	discover relevant work and scope an extension	suggestion presented as a claim made by the source paper
Researcher	improve discoverability and produce accessible summaries	inaccurate or decontextualised representation
Public/industry user	connect a problem to papers or potential collaborators	expertise inferred from noisy authorship/topic data
Administrator	correct metadata and approve generated outputs	unauthenticated or incomplete review workflow
Evaluator	judge retrieval, grounding, and usefulness	structural checks mistaken for empirical quality

ment, unrestricted crawling, optical character recognition, learned topic modelling, trained recommendation models, external OpenAI generation, and audio synthesis are outside the implemented scope.

The intended institutional identity and corpus boundary still require confirmation: **AUTHOR INPUT REQUIRED: confirm the official laboratory name, acronym expansion, and whether the 134-record seed is an exhaustive date-bounded snapshot.**

2.3 Success criteria

For this thesis, implementation success means that the artefact can be built and exercised reproducibly, retains page/chunk provenance, reports failure states, and exposes generated material for review. Empirical success would additionally require labelled retrieval cases, answer and citation judgments, recommendation review, participant or expert protocol, and appropriate analysis. The first can be established from the repository; the second has not yet been established.

Research Questions and Objectives

The research questions are framed so that the current repository can answer implementation and characterization questions without manufacturing effectiveness results.

- RQ1** With what coverage and observable failure modes can the publication snapshot be converted into page-aware full-text chunks?
- RQ2** How does the implemented retrieval and question-answering workflow preserve traceability under the currently active index?
- RQ3** How does the thesis-extension workflow distinguish source-supported baseline work and gaps from system-generated suggestions?
- RQ4** How are topics, authors, metadata, and generated outputs made inspectable and reviewable, and what data-quality risks remain?
- RQ5** To what extent can the prototype and its current evidence be reproduced and audited, and what evaluation is still required before effectiveness claims?

3.1 Question-to-evidence design

The full evidence-result-conclusion matrix appears in [appendix A](#). This structure prevents a passing software test from being used as evidence for a user-facing quality claim.

3.2 Propositions not yet tested

The following plausible propositions are intentionally not converted into conclusions: hybrid retrieval improves relevance over keyword search; page citations improve trust; extension recommendations help students choose feasible work; and administrative review improves public-output quality. Testing them requires labelled baselines and/or human participants.

AUTHOR INPUT REQUIRED: confirm whether these should become formal hypothe-

Table 3.1: Research-question traceability plan.

RQ	Method	Evidence	Bounded conclusion type
1	repository and corpus audit	seed, PDF/text/chunk manifests, extraction diagnostics, SQLite counts	coverage and observed ingestion limitations
2	code inspection and disposable runtime probes	retrieval code, active index, search and Ask responses, citation verifier	traceability mechanics, not answer accuracy
3	schema/algorithm inspection and recommendation probe	weights, evidence fields, verifier, persisted/run-time outputs	structural separation, not educational usefulness
4	database/UI inspection	topic and author links, review API, screenshots, review-event count	inspectability and defects, not validated expertise
5	isolated tests, builds, hashes, build artefacts	test/build logs, scripts, generated snapshots, PDF QA	reproducibility of the audited snapshot

ses and supply the approved evaluation protocol.

Literature Review

4.1 Scholarly discovery and research intelligence

Early autonomous citation indexing showed how web-accessible papers could be gathered and linked into searchable digital libraries [2]. ArnetMiner combined extraction and mining to support expert and topic discovery [3], while S2ORC and OpenAlex demonstrate the value of large scholarly graphs and open metadata [4, 5]. The TTLAB precursor focused on collecting and presenting publications, producing lay summaries, notifications, and podcast material [1]. The current artefact differs principally in its use of full local papers, page-aware chunks, student-facing extension structures, and explicit review state.

This local focus creates a trade-off. Large scholarly systems offer scale and citation networks; the prototype can retain inspectable full-text provenance and domain-specific workflows, but inherits the incompleteness and noise of its small corpus.

4.2 Scholarly recommendation and expertise discovery

Research-paper recommendation has used content, citations, user profiles, and hybrid signals; surveys identify evaluation comparability and realistic user modelling as persistent difficulties [7]. Scholarly recommendations can also exploit a researcher's prior publications to infer interests [8]. Expert finding is commonly formalised as ranking candidates from textual evidence, including candidate-centric and document-centric language models [9, 10].

The project does not implement a learned recommender or probabilistic expert model. It uses deterministic weighted rules and labels author expertise as potential fit. This distinction matters because authorship plus lexical topics is supporting evidence for a possible connection, not proof of present expertise, availability, or willingness to collaborate.

4.3 Lexical and dense retrieval

BM25 remains a strong lexical retrieval baseline [11]. Learned sentence and passage representations, including Sentence-BERT, Dense Passage Retrieval, and SPECTER, enable semantic similarity beyond exact term overlap [12–14]. BEIR shows that zero-shot retrieval performance varies substantially across tasks and that no single method should be assumed superior without evaluation [15].

The prototype combines SQLite FTS ranking with a 256-dimensional signed feature-hashing representation. Feature hashing is a memory-efficient projection technique [16]; unlike learned dense encoders, it mainly preserves lexical features. Calling the active path “semantic” in the interface is therefore looser than the algorithm warrants. The thesis uses the term *hashing-vector retrieval*.

4.4 Topic modelling and author-topic relations

Probabilistic topic models such as latent Dirichlet allocation infer distributions over latent topics [17]; BERTopic combines transformer embeddings with clustering and class-based term weighting [18]. Topic-model evaluation remains difficult because numerical coherence and human interpretability can diverge [19]. The project instead applies a fixed lexical taxonomy of 39 rules. This improves determinism and domain editability, but does not discover latent topics and has not been validated for coverage or agreement.

4.5 Retrieval-augmented generation and grounding

RAG conditions generation on retrieved documents and can expose provenance [6]. Fusion-in-Decoder aggregates multiple passages in the generator [20], while Self-RAG adds retrieval and self-reflection control tokens [21]. These methods motivate retrieval-aware answer construction, but do not imply that any system returning citations is faithful.

Frameworks such as RAGAS and ARES propose automated dimensions and synthetic evaluation for RAG pipelines [22, 23]. FActScore decomposes long-form generations into atomic factual claims [24]. For summaries, SummEval demonstrates the need to compare automatic measures with human judgments [25]; G-Eval explores model-based evaluation but remains dependent on prompt and evaluator behaviour [26]. The present verifier checks citation structure and token overlap only. It is properly treated as a guardrail, not a faithfulness metric.

4.6 Human review and responsible AI

Interactive machine learning emphasizes that effective systems incorporate human feedback at appropriate points in the workflow [27]. Dataset documentation and model reporting practices make provenance, intended use, and limitations explicit [28, 29]. Language-model risk taxonomies include discrimination, misinformation, malicious use, and human-computer interaction harms [30]; the NIST AI Risk Management Framework similarly emphasizes governance, mapping, measurement, and management [31].

The platform’s `needs_review` state, warning fields, evidence links, and review-event model align with these principles at the mechanism level. However, zero recorded review events and the absence of authentication mean that review effectiveness and governance are not established.

4.7 Research gap

The reviewed literature offers mature components for scholarly graphs, recommendation, expert finding, retrieval, topic modelling, RAG, and evaluation. The practical gap addressed here is their integration into a small-laboratory workflow that connects full-paper evidence to student project formulation and public explanation while keeping source facts, inferences, suggestions, and review state separate. The contribution is integration and auditability in this context, not a replacement for the large-scale or learned methods above.

Methodology

5.1 Study design

This work is reported as an artefact-oriented engineering case study with design-science characteristics: a problem-motivated software artefact is built, demonstrated, and evaluated through repository and runtime evidence. **AUTHOR INPUT REQUIRED: confirm that this methodological designation matches the approved MSc proposal and university terminology.** The present thesis evaluation is a technical characterization, not a completed user study.

5.2 Evidence protocol

The audit used six evidence classes, ordered by what each can establish:

1. source and schema inspection establishes implemented logic and data contracts;
2. file and database manifests establish snapshot coverage and consistency;
3. isolated tests establish tested behaviour under test assumptions;
4. disposable runtime probes establish endpoint behaviour without contaminating the project database;
5. frontend builds and browser inspection establish client compilation and visible integration; and
6. literature records establish externally verifiable context, not project performance.

Claims were rejected or labelled as requiring author input when no corresponding evidence existed. Commit subjects were used as chronology, never as sole proof of functionality. The ignored runtime database was hashed before probing, and API probes operated on a temporary copy.

5.3 Reproducible snapshot collection

The script `thesis/scripts/collect_evidence.py` records Git state, file hashes, corpus manifests, SQLite integrity, table counts, chunk sections, semantic-index coverage, and evaluation artefacts. It emits JSON and CSV files consumed by the thesis figures. The script is read-only with respect to the project database. `thesis/scripts/runtime_probe.py` copies that database to a temporary location, sets an isolated database URL, and issues representative requests through FastAPI's test client.

5.4 Analytical boundaries

The reported counts are descriptive statistics over one dated repository snapshot. No sampling uncertainty is attached to counts such as PDFs or chunks. Retrieval scores from illustrative requests are not aggregated effectiveness metrics. A structurally grounded answer is not counted as factually correct. Placeholder evaluation cases are not treated as experiments. These boundaries are essential because an evaluation dashboard can exist while evaluation data do not.

5.5 Proposed empirical methodology

The implemented evaluator supports binary paper-relevance labels and reciprocal rank. A defensible follow-up study should:

1. freeze a versioned corpus and index manifest;
2. have at least two domain-informed assessors label relevant papers and answer points for a stratified set of fact, method, limitation, comparison, topic, author, and broad-intent questions;
3. compare keyword, hashing-vector, and hybrid retrieval under identical cut-offs;
4. review answer claims and citations for entailment, completeness, and usefulness;
5. evaluate recommendation relevance, feasibility, novelty, source fidelity, and calibration with students and/or supervisors; and
6. report agreement, uncertainty, failures, latency, hardware, model version, prompts, repetitions, and statistical tests appropriate to the final design.

Until that protocol is approved and executed, the thesis reports only implementation and reproducibility results. **AUTHOR INPUT REQUIRED: provide ethics approval/exemption status, assessor recruitment plan, consent procedure, sample-size rationale, and**

approved statistical analysis.

System Architecture and Design

6.1 Architectural overview

The system follows a four-layer local architecture (Fig. 6.1). The React presentation layer calls a FastAPI interface. Application services implement ingestion, retrieval, generation, recommendation, exploration, review, and evaluation. SQLite and local files retain structured records, source documents, derived text, indexes, and evaluation artefacts.

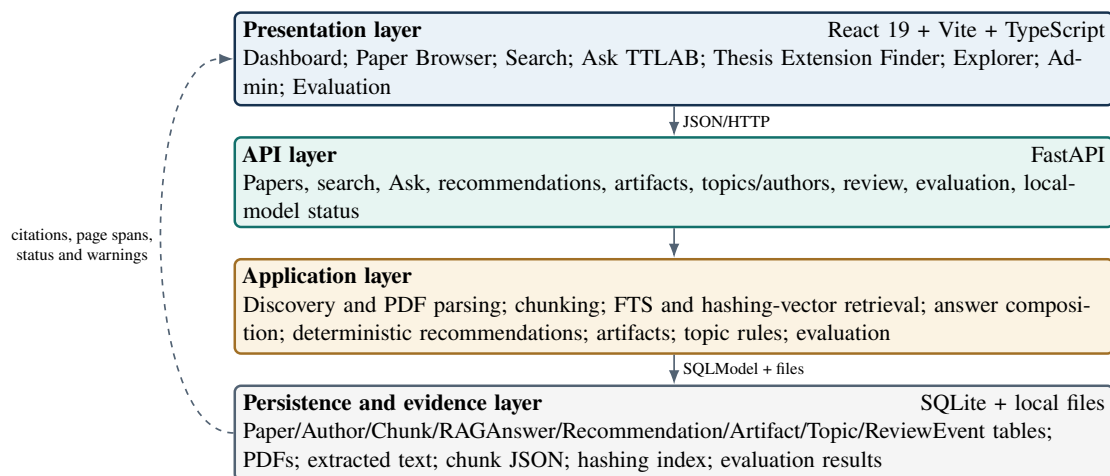


Figure 6.1: Implemented system architecture and evidence return path. Dashed flow emphasizes provenance and status returned to the interface.

This organization supports local reproducibility and failure inspection. It also creates consistency risks because derived state is split between SQLite and JSON files. The active hashing index and the database embedding-status field illustrate that risk in [chapter 10](#).

6.2 Entity and provenance design

Figure 6.2 shows the central entities. A chunk retains paper, page span, section label, text, hash, and embedding state. Answers, recommendations, and artefacts retain JSON-encoded citations and generated structures. Review events refer polymorphically to reviewed items and record a status transition and field diff.

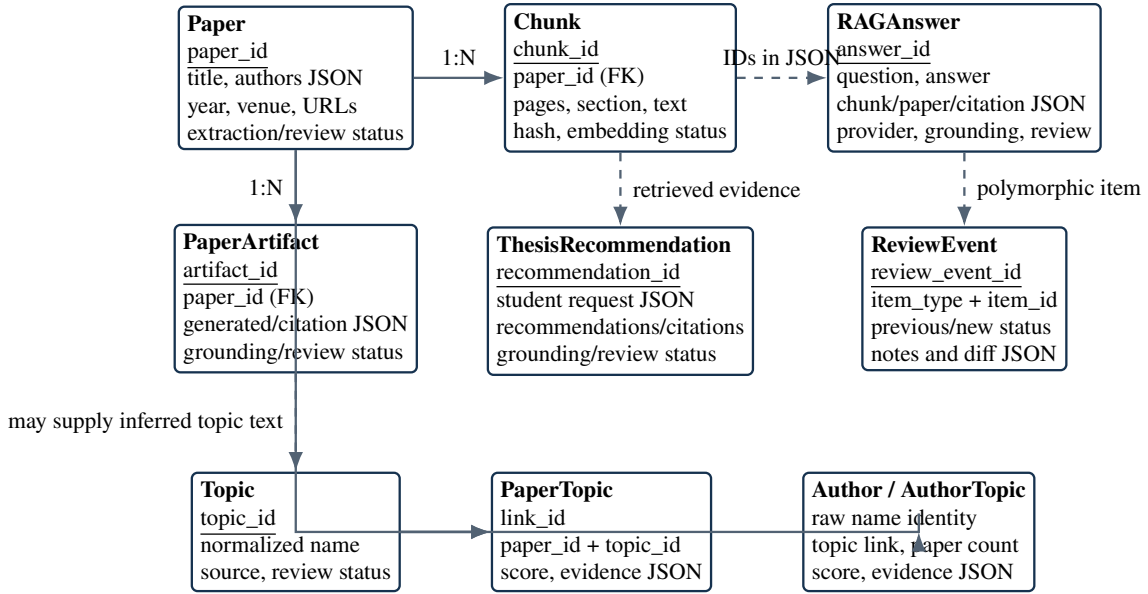


Figure 6.2: Simplified entity and provenance model reconstructed from the SQLModel classes. JSON fields are summarized rather than expanded.

JSON-encoded columns simplify a local prototype but weaken relational constraints over citation membership and nested recommendation structures. Application verifiers consequently check relationships that a normalized schema might otherwise enforce.

6.3 Traceability as a cross-cutting requirement

Traceability is carried through four identifiers: paper ID, chunk ID, page span, and generated-item ID. Generation outputs additionally store provider/model metadata, status, warnings, timestamp, and review status. The design separates:

- *source facts*: extracted text and metadata;
- *retrieval decisions*: ranked chunks and scores;
- *system interpretation*: extractive answers, inferred gaps, topics, or summaries;
- *system suggestions*: extension scope, skills, risk, or evaluation plan; and
- *human decision*: review status, notes, and field changes.

The distinction exists in the data contracts, but the current database contains no completed human review events. Traceability infrastructure must therefore not be confused with actual governance.

6.4 Operational architecture

Configuration is environment-driven. A shell orchestrator can prepare a bounded demo, verify dependencies, and start FastAPI and Vite; the local model is optional and errors fall back to offline extraction. There is no production deployment manifest, reverse proxy, authentication, role-based authorization, or versioned schema-migration framework. The operational architecture is a development/demo architecture.

6.5 Design decisions and alternatives

SQLite and local files were chosen for a short, curated project rather than a distributed database and vector service. Deterministic IDs and hashing improve repeatability. Page-aware chunks improve inspectability compared with abstract-only records. The trade-offs are limited concurrency, split derived state, heuristic structure inference, and lexical rather than learned vector similarity. Learned encoders such as Sentence-BERT or SPECTER are relevant alternatives [12, 14], but the corresponding provider branches are not operational in this snapshot.

Data Collection and Processing

7.1 Corpus provenance

The seed contains 134 publication records derived from a TTLAB WordPress archive and matches the saved discovery JSON byte-for-byte. Its SHA-256 digest is recorded in PROJECT_EVIDENCE.md. The snapshot includes local PDFs for 98 papers; 35 are missing and one download is marked failed. **AUTHOR INPUT REQUIRED: confirm the collection date, archive completeness, laboratory authorization, and copyright/licensing basis for retaining PDFs and displaying snippets.**

7.2 Ingestion workflow

Figure 7.1 reconstructs the implemented path. Discovery parses archive metadata and stable identifiers. Manual import upserts paper and raw author records. The downloader accepts direct PDF-like URLs and checks content type and the PDF signature. PyMuPDF extracts text page by page and records counts, empty pages, a text hash, and low-text warnings. A sentence heuristic then forms page-aware, overlapping chunks.

7.3 Chunking

The default chunk target is 850 words with a maximum of 5,000 characters and 125 overlapping words. A final chunk shorter than 400 characters is merged with its predecessor. Each identifier incorporates paper ID, sequence, page span, and content hash. Sections are inferred by matching headings within the first 1,200 characters of a chunk; no document-layout model is used.

The 98 extracted papers contain 698 pages, 455,601 source words, and 756 chunks. Overlap raises the sum of chunk words to 547,628, a ratio of 1.202. Of 756 chunks, 249 (32.94%) have

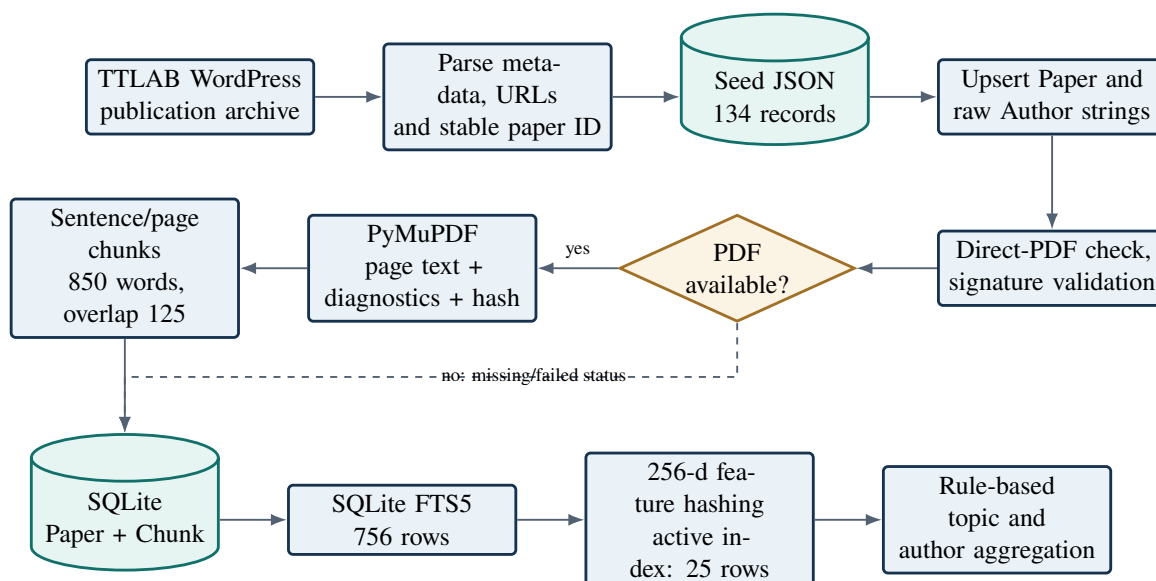


Figure 7.1: Implemented ingestion, extraction, indexing, and topic-processing pipeline with snapshot counts.

section Unknown. The dominant recognized sections are Methodology (242) and Results (102). Section-sensitive reranking and artefact generation therefore operate over incomplete structural labels.

7.4 Corpus status

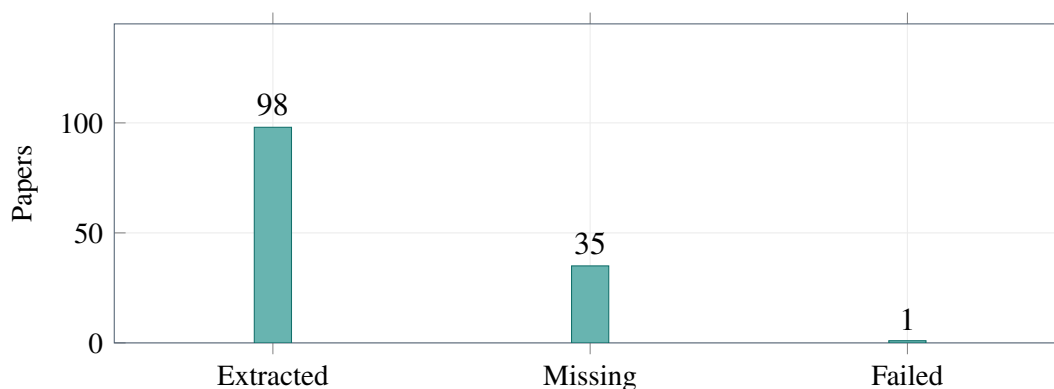


Figure 7.2: Paper-level PDF/extraction status in the audited database. Counts sum to the 134 seed records.

The 98 database PDF paths map one-to-one to extracted JSON, extracted text, and chunk JSON in the current snapshot. The sorted PDF checksum manifest covers 29,718,138 bytes. SQLite integrity and foreign-key checks passed. These checks establish snapshot consistency, not text-extraction correctness.

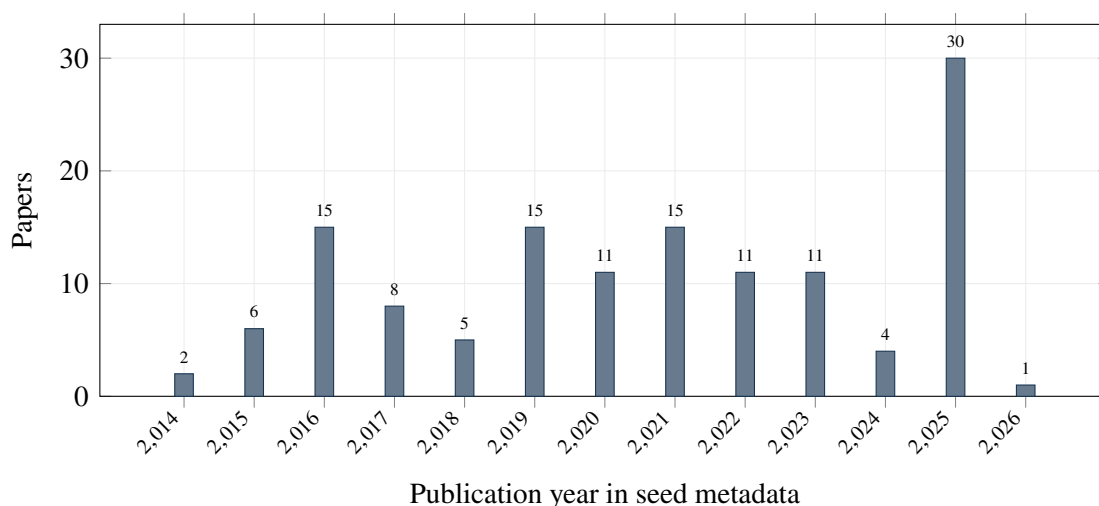


Figure 7.3: Distribution of publication years in the seed metadata. Values are descriptive of the audited snapshot, not a publication-rate analysis.

7.5 Metadata quality

All papers require review. No structured DOI, abstract, keyword, or denormalized paper-topic field is populated in the database; normalized topics are stored separately. Six venues and 93 source URLs are missing. Raw author strings yield 126 records, including separate identities for “Patrick Hosein” and “P. Hosein”, and the parsing artefact “Click to View” is linked to 12 papers. Counts of authors and expertise connections must consequently be interpreted as unvalidated string-level records.

7.6 Data minimization and retention

The system stores publication documents and public scholarly metadata rather than user accounts. Recommendation requests can nevertheless contain interests, skills, and constraints; persisted requests may be personal data. No retention schedule, consent notice, access control, or deletion workflow is implemented. Public deployment requires decisions on these issues before recommendation inputs are retained.

Implementation

8.1 Technology stack and interface

The backend uses Python, FastAPI, SQLAlchemy, Pydantic settings, SQLite, and PyMuPDF. The frontend uses React, TypeScript, and Vite. Eight views expose dashboard statistics, papers, search, Ask TTLAB, extension recommendations, topic/author exploration, administrative review, and evaluation. Figures 8.1 and 8.2 show the audited runtime.

Navigation is held in React state rather than URL routes, so refreshing returns to the dashboard and views are not deep-linkable.

8.2 Keyword and hashing-vector retrieval

SQLite FTS5 indexes all chunk text and returns its `bm25()` rank, transformed by

$$K = \frac{1}{1 + |r_{\text{FTS}}|}. \quad (8.1)$$

A deterministic fallback combines term frequency and query coverage when FTS is unavailable.

The vector path maps token t into one of $d = 256$ coordinates and a sign using SHA-256. For document x , the unnormalised coordinate is

$$z_j(x) = \sum_{t \in x} \mathbb{I}[h(t) = j] s(t), \quad \mathbf{v}(x) = \frac{\mathbf{z}(x)}{\|\mathbf{z}(x)\|_2}. \quad (8.2)$$

Similarity is the dot product $V = \mathbf{v}(q)^\top \mathbf{v}(c)$, equivalent to cosine similarity after normalisation. This is feature hashing [16], not a learned semantic encoder.

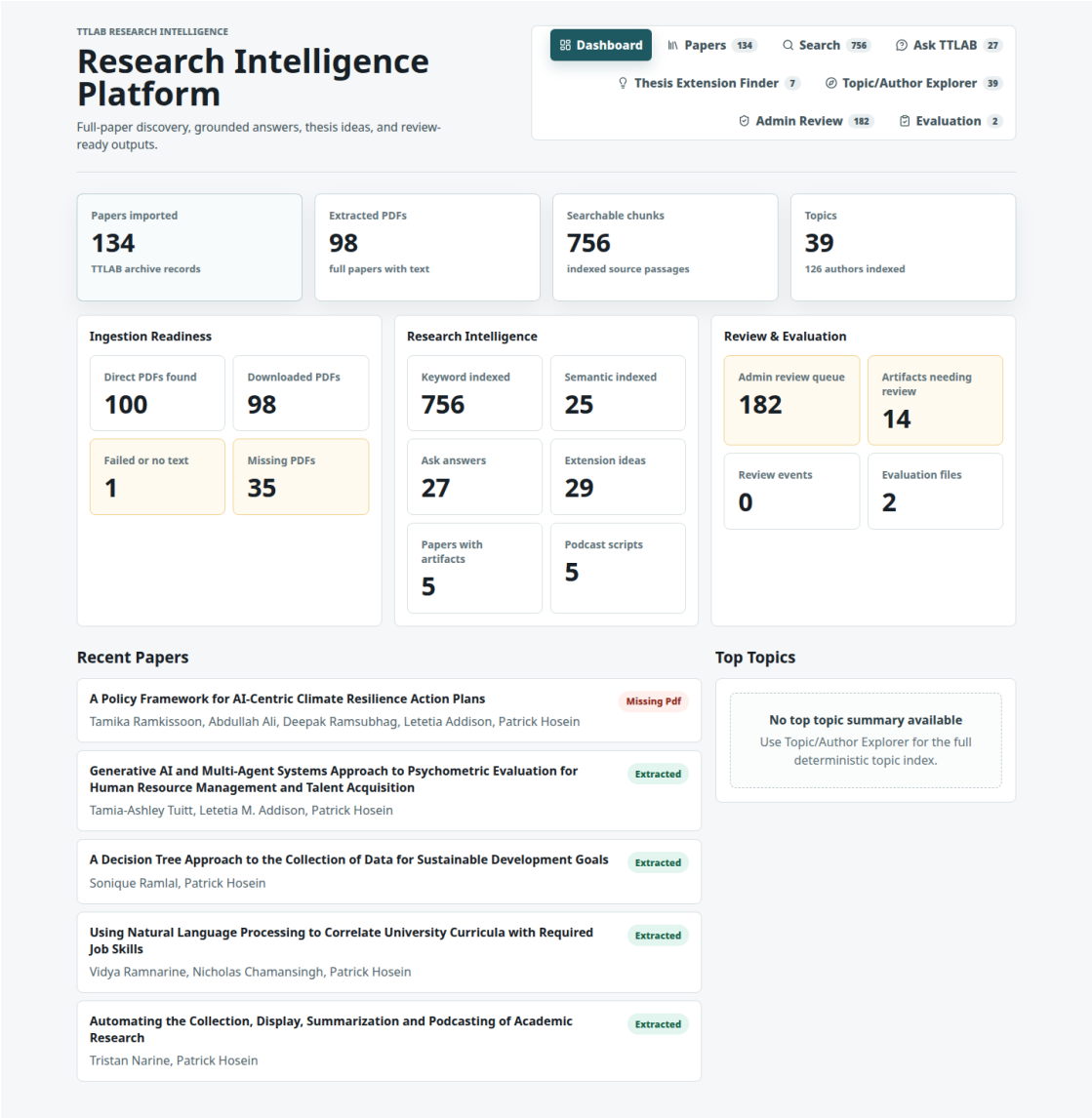


Figure 8.1: Dashboard in the audited browser session. The empty top-topic widget contrasts with populated explorer data and is retained as failure evidence.

TTLAB RESEARCH INTELLIGENCE

Research Intelligence Platform

Full-paper discovery, grounded answers, thesis ideas, and review-ready outputs.

Dashboard Papers 134 Search 756 Ask TTLAB 27

Thesis Extension Finder 7 Topic/Author Explorer 39

Admin Review 182 Evaluation 2

Search Source Chunks

RAG academic research Hybrid 10 Search

756 searchable chunks 756 keyword indexed 25 semantic indexed hashing - ready

Rank 1 Unknown Pages 3-4 Score 0.268

Using Retrieval-Augmented Generation for Fault Prediction and Maintenance on the Factory Floor

Adir Soechit, Patrick Hosein · 2025

assified as 'unusual patterns' (LSTM detected, but within defined thresholds). D. [[RAG]]-enabled LLM Reasoning Core This core provides intelligent diagnostics by combining LLM capabilities with external knowledge. Upon anomaly detection, a deta

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 2 Methodology Pages 4-6 Score 0.265

A Comparison of Large Language Models in a Retrieval-Augmented Generation (RAG) System for Machine Repair and Maintenance Prompts

Shane Tikasingh, Patrick Hosein · 2025

y better than the llama3.2:3b ($\Delta = 0.01$). V. DISCUSSION This study constructed a [[RAG]] system for industrial tech- nical manuals and methodically evaluated the performance of five open-source LLMs across four different question types. Fig. 4. M

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 3 Results Pages 3-4 Score 0.207

A Comparison of Large Language Models in a Retrieval-Augmented Generation (RAG) System for Machine Repair and Maintenance Prompts

Shane Tikasingh, Patrick Hosein · 2025

is then generated by the LLM. 3) Evaluation: Evaluating the performance of the [[RAG]] model requires a systematic comparison between reference answers and candidate answers. This section outlines the methodology for generating reference quest

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 4 Literature Review Pages 1-2 Score 0.187

Using Retrieval-Augmented Generation for Fault Prediction and Maintenance on the Factory Floor

Adir Soechit, Patrick Hosein · 2025

The objective is to design, implement and rigorously evaluate a [[RAG]]-based LLM system, integrated with specialized monitoring and communication agents, tailored for real-time anomaly detection and comprehensive predictive maintenance in Indu

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 5 Discussion Pages 4-5 Score 0.152

A Global Hierarchical LLM Framework for Precise Responses through Subscriptions to Country LLMs

Anthony Jairam, Tamika Ramkissoon, Kevin Baboolal, Patrick Hosein · 2025

or- chestration framework realizes it's need for additional context, and the DS-[[RAG]] network is invoked to resolve the query. ChatGPT first pings the SIDS Registry Node to determine which Sovereign Node will be best equipped to handle the que

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 6 Methodology Pages 2-3 Score 0.119

A Comparison of Large Language Models in a Retrieval-Augmented Generation (RAG) System for Machine Repair and Maintenance Prompts

Shane Tikasingh, Patrick Hosein · 2025

Taken together, these studies illustrate a trajectory in [[RAG]] research that moves from optimizing model-level components (LLMs, embeddings, conversation strategies) and inventing novel [[retrieval]] paradigms toward engineering and evaluating fu

[PDF](#) [TTLAB post](#) [Paper detail](#)

Rank 7 Evaluation Pages 5-6 Score 0.088

Search returned 10 source chunks.

Figure 8.2: Search results with mode, scores, paper metadata, pages, and source snippets. The full-page capture is deliberately cropped to its top results for legibility.

8.3 Hybrid scoring and diversification

Current working-tree code removes a fixed stop-word set, applies fixed domain query expansions, and computes

$$H = \text{clip}_{[0,1]}(0.42K + 0.48V + M + S + E + T), \quad (8.3)$$

where M is metadata match, S an intent-dependent section boost, E an evidence-quality adjustment, and T topical alignment. A greedy penalty reduces repeated or adjacent chunks from the same paper. The coefficients are hand-authored; no tuning or ablation evidence exists.

8.4 Question answering

The workflow in Fig. 8.3 retrieves chunks, composes an answer with a deterministic offline provider or local Ollama, attaches citations, verifies structural properties, persists the result, and displays the answer with source pages and warnings.

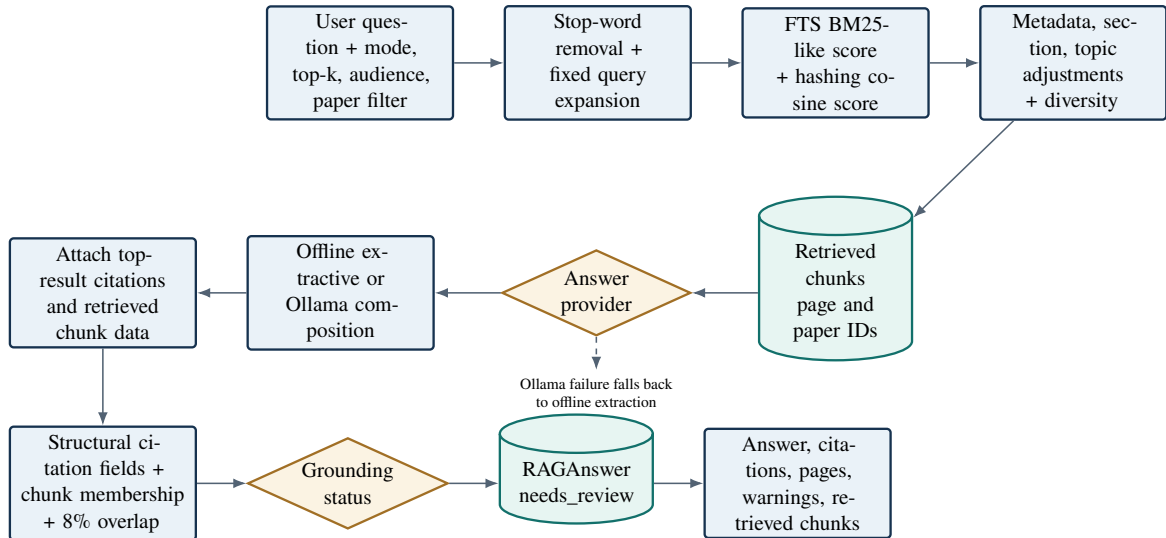


Figure 8.3: Implemented search and RAG workflow. The verifier checks structure, membership, and lexical overlap rather than semantic entailment.

The Ollama adapter uses a low-temperature, source-restricted prompt and falls back on error. The OpenAI provider branch raises a not-implemented error. The verifier requires retrieved chunks, complete citation fields, citation membership, and at least 0.08 answer-to-snippet token overlap. Therefore, its grounded label means structurally/lexically grounded only.

The screenshot displays the TTLAB Research Intelligence Platform interface. At the top, the header includes the platform name and a navigation bar with links to Dashboard, Papers (134), Search (756), Ask TTLAB (27), Thesis Extension Finder (7), Topic/Author Explorer (39), Admin Review (182), and Evaluation (2).

The main section is titled "Ask TTLAB" and contains a search input field with the query "Which TTLAB papers discuss RAG?". Below the input field are several filters: General/public, Hybrid, Top 5, All indexed papers, Offline extractive, and a model selector set to qwen3:4b-instruct. An "Ask" button is located to the right of the filters.

Below the filters, the interface shows search statistics: 756 searchable chunks, 27 stored answers, and a status for Ollama (Ollama unavailable). A recommendation section highlights "qwen3:4b-instruct-2507-q4_K_M" as the best default candidate.

A message indicates that Ollama is not reachable at the specified URL. Below this, the "Grounded" state is shown, indicating that the answer is generated from indexed TTLAB paper chunks and should be checked against the cited sources.

The "Answer" section provides a detailed response based on the indexed paper chunks, mentioning the strongest matching papers and their authors. It also includes a "Query expansion" section with related terms like "retrieval augmented generation" and "grounded generation".

Two notification boxes are visible: "Search returned 10 source chunks." and "Ask TTLAB answered with 5 citations."

The "Citations" section lists two papers:

- A Comparison of Large Language Models in a Retrieval-Augmented Generation (RAG) System for Machine Repair and Maintenance Prompts** by Shane Tikasingh, Patrick Hosein (2025). The paper discusses a trajectory in RAG research, from optimizing model-level components to engineering and evaluating full systems in applied settings.
- Using Retrieval-Augmented Generation for Fault Prediction and Maintenance on the Factory Floor** by Adhir Soechit, Patrick Hosein (2025). The paper describes an innovative RAG system for real-time anomaly detection and predictive maintenance, leveraging Large Language Models (LLMs).

Figure 8.4: Ask TTLAB response showing generated-answer notice, grounding state, and cited papers. The full-page capture is deliberately cropped to the answer and leading citations.

8.5 Thesis-extension recommendations

Figure 8.5 shows the deterministic workflow. Candidate paper score R is

$$R = 0.30r + 0.20i + 0.15s + 0.10d + 0.10t \quad (8.4)$$

$$+ 0.10f + 0.05e - 0.20a, \quad (8.5)$$

where the terms represent retrieval relevance, interest, skill, data, time, difficulty fit, evidence strength, and avoided-topic match. The output distinguishes an explicit, inferred, or absent source gap from the system’s suggested extension.

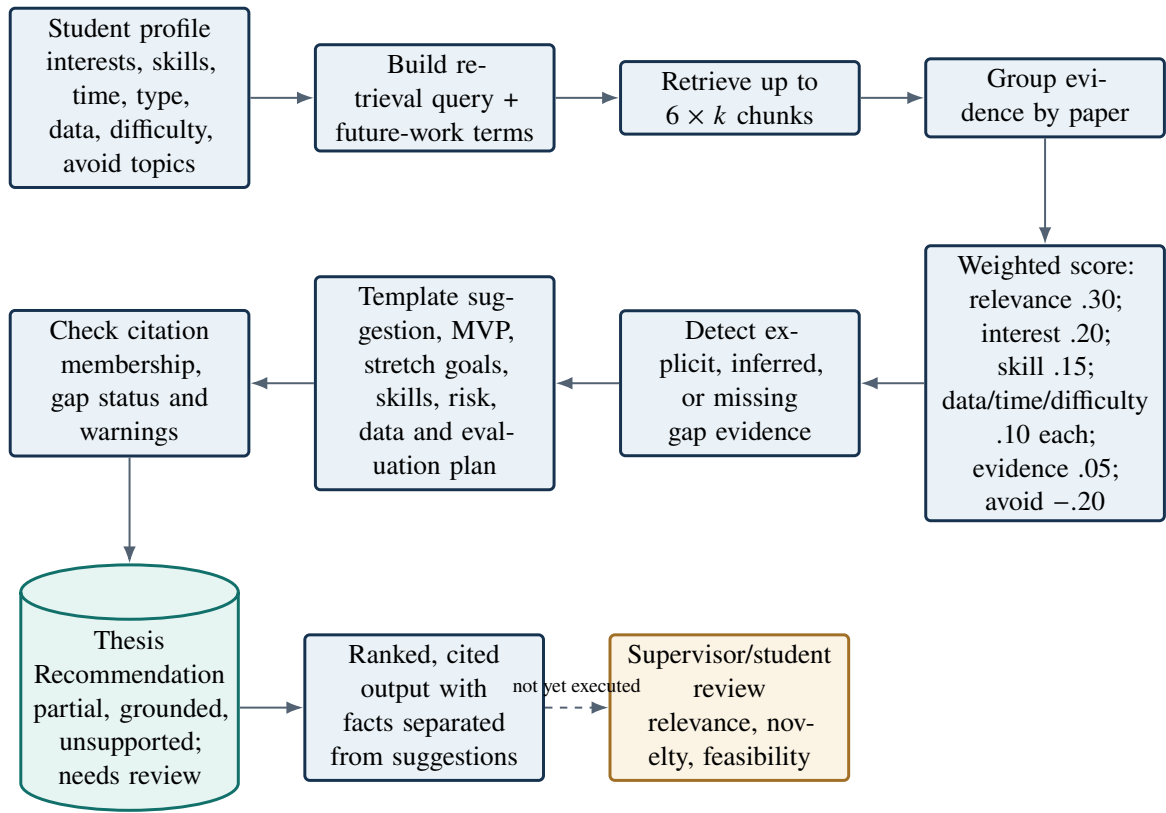


Figure 8.5: Implemented recommendation pipeline and the human-validation stage that has not yet been executed.

8.6 Topic, author, artefact, and review functions

Topic discovery uses a fixed 39-topic lexical taxonomy and weighted text sources. It can retain up to eight topic links per paper; author-topic scores aggregate paper topics. Related-paper scoring combines topics, authors, venue, year, lexical overlap, and hashing-vector signals. This is rule-based classification, not topic modelling.

The paper-intelligence generator uses section and lexical priorities to create public and tech-

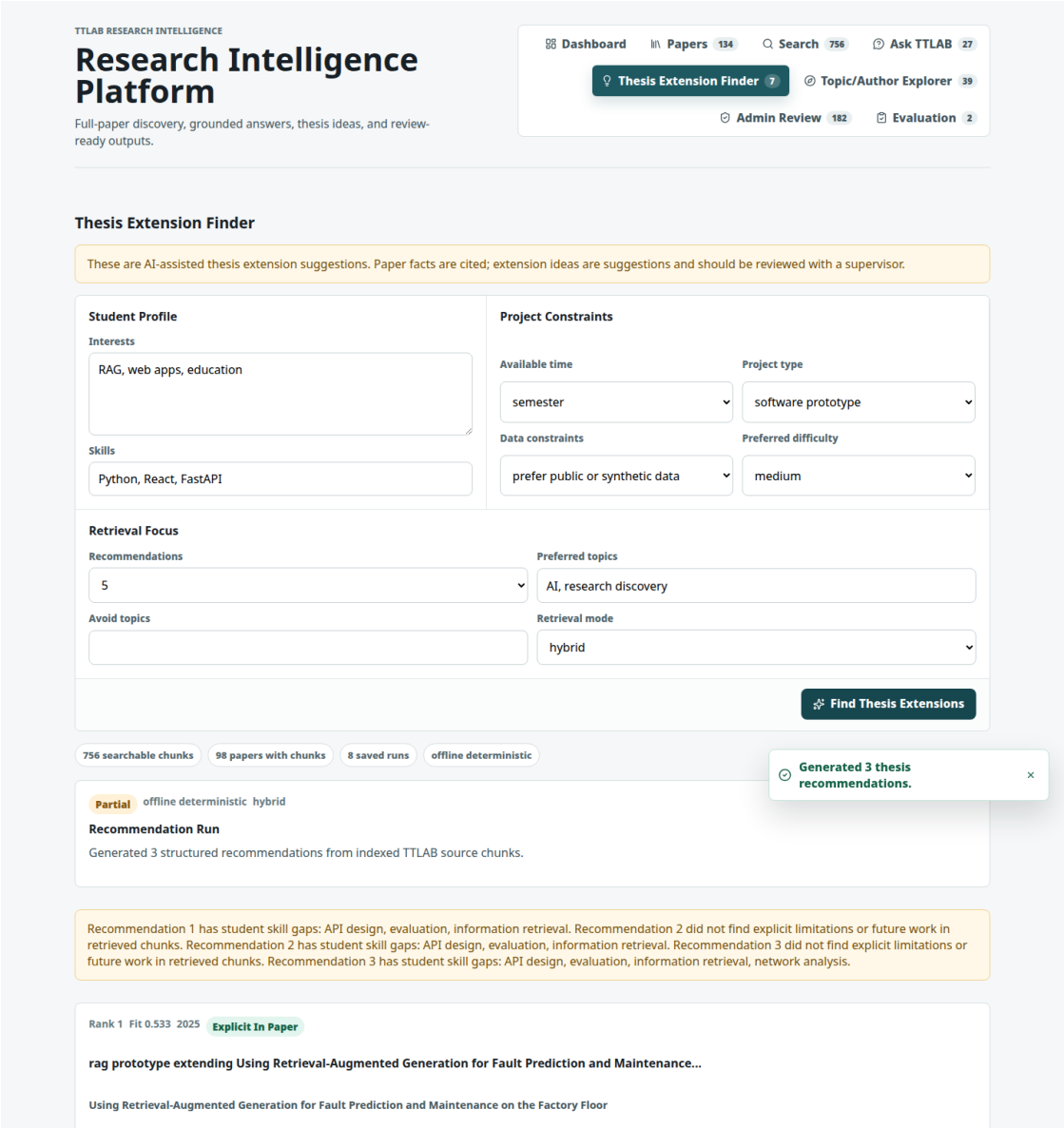
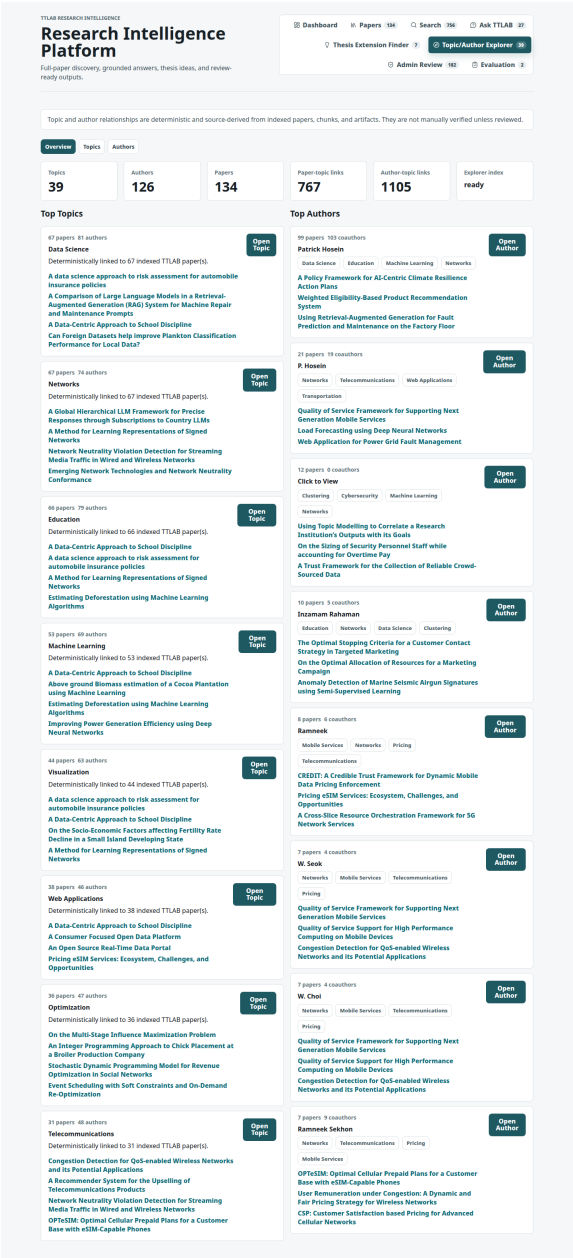


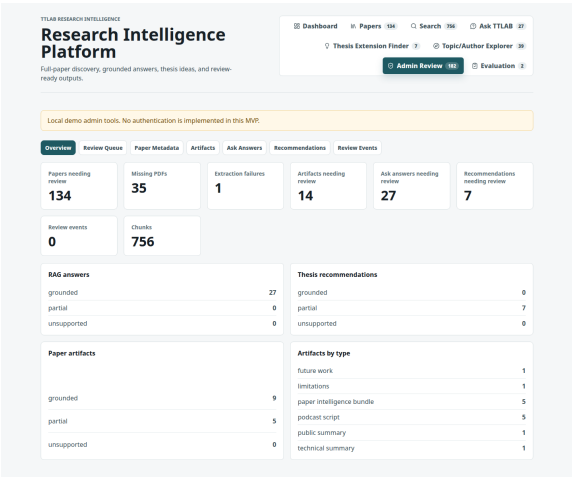
Figure 8.6: Extension Finder profile and leading ranked result with source/gap status. The full-page capture is deliberately cropped before additional result detail.

nical summaries, contributions, methods, limitations, future work, suggestions, skills, evaluation ideas, and text-only podcast dialogue. It labels limitations/future work as explicit, inferred, or not found and labels extensions as system suggestions.

Review endpoints can update metadata and statuses and write a review event. The UI exposes queues and status controls, but there is no authentication; the reviewer identity is fixed to local_admin. Figures 8.7a and 8.7b show both the explorer’s visible identity defect and the review mechanism.



(a) Topic/author explorer.



(b) Administrative review queue.

Figure 8.7: Discovery and review interfaces. The explorer screenshot retains the “Click to View” author artefact.

Experimental Design

9.1 Completed technical verification

Completed verification comprises source/schema audit, file and database consistency checks, automated tests against an isolated database, representative API probes against a disposable database copy, a clean frontend dependency/build run, browser inspection of eight main surfaces, and LaTeX build/render inspection. These procedures answer whether the snapshot is internally reproducible and demonstrable; they do not answer whether users receive relevant or faithful advice.

9.2 Implemented retrieval metrics

For query q , with retrieved paper list L_q and gold set G_q , the code reports a binary hit indicator called $\text{Recall}@k$:

$$\text{Hit}@k(q) = \mathbb{I}[G_q \cap L_q[1:k] \neq \emptyset]. \quad (9.1)$$

The implementation averages this indicator over queries. It is therefore a hit rate, not set recall $|G_q \cap L_q[1:k]|/|G_q|$. Mean reciprocal rank is

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (9.2)$$

with zero when no gold paper is retrieved. The sample retrieval and QA files contain empty gold labels and intentionally fail validation, so neither metric has a current result.

9.3 Proposed retrieval experiment

A versioned test set should stratify questions by exact fact, method, result, limitation/future work, topic, author, comparison, and broad intent. Keyword, hashing-vector, and hybrid modes should use the same corpus, cut-offs, and paper filtering. Primary outcomes should include set Recall@3/5, hit rate, MRR, nDCG where graded judgments are available, and per-intent error analysis. Query-expansion and reranking terms should be ablated. The active vector index must first be rebuilt for all 756 chunks and status consistency verified.

9.4 Proposed QA evaluation

Each answer should be decomposed into checkable claims. Reviewers should label whether each claim is entailed by its cited snippet, whether the citation points to the right paper/page, whether important answer points are covered, and whether unsupported general knowledge appears. Automatic RAG measures may supplement but not replace these labels [22–24]. Provider, model hash/tag, prompt, temperature, retrieval list, and fallback state must be stored.

9.5 Proposed recommendation evaluation

Students and supervisors should independently rate paper relevance, gap fidelity, novelty, feasibility for the stated time/skills/data, usefulness of the MVP, risk calibration, and appropriateness of the evaluation plan. Reviewers must separately judge source claims and system suggestions. A comparison baseline could present retrieved papers without generated extension advice. **AUTHOR INPUT REQUIRED: approve participant groups, sample size, rating scales, compensation, consent, and analysis.**

9.6 Proposed topic/author and summary evaluation

Topic evaluation should measure label precision/coverage and reviewer agreement against the fixed taxonomy, including multi-label cases and an “other” outcome. Author evaluation should first resolve identities, then assess expertise links. Public and technical summaries should be reviewed for correctness, relevance, coherence, and readability, drawing on human-centred evaluation lessons from SummEval [25].

9.7 Evaluation workflow and current state

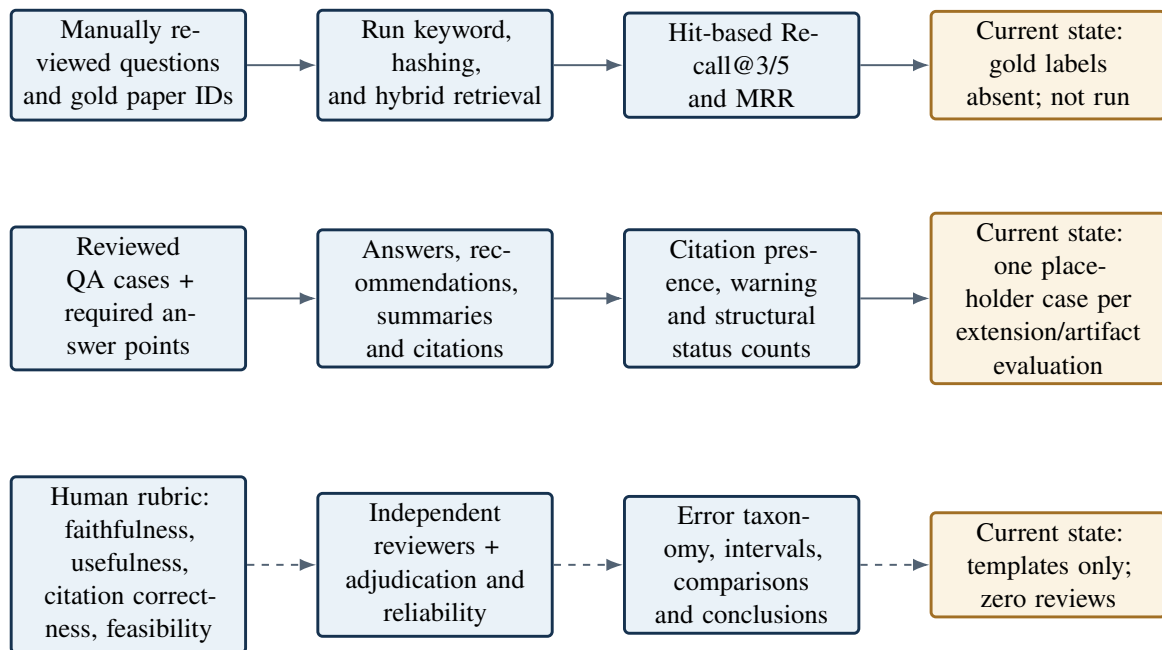


Figure 9.1: Implemented and proposed evaluation workflow. Gold retrieval/QA labels and human reviews are absent in the audited snapshot.

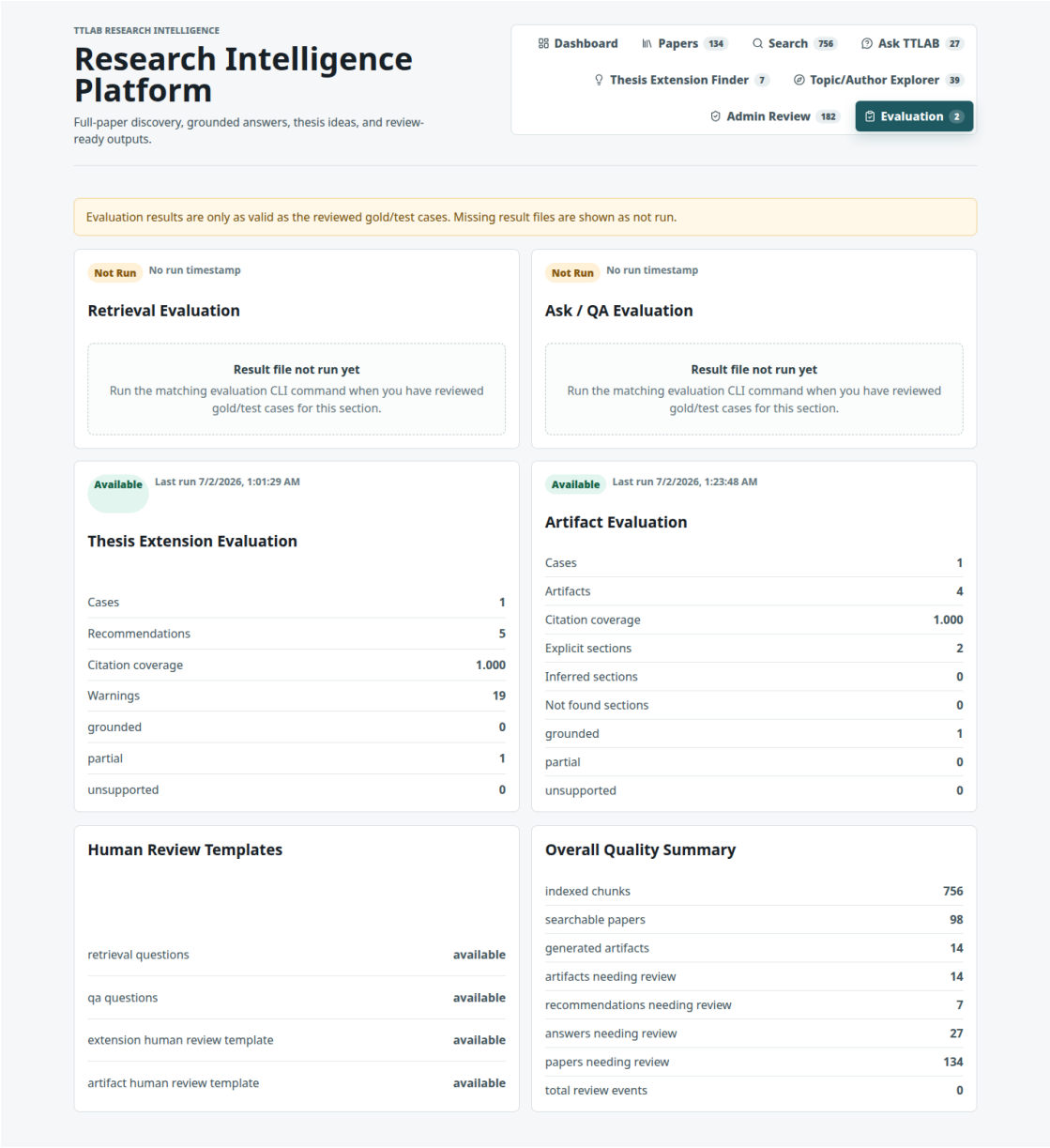


Figure 9.2: Evaluation dashboard correctly displaying not-run and placeholder states rather than inferred performance.

Results

10.1 Corpus and ingestion results (RQ1)

The pipeline represents all 134 seed records in SQLite and has extracted 98 PDFs into 698 pages and 756 chunks. Thirty-five papers lack PDFs and one is marked download failed (Fig. 7.2). PDF, extracted JSON/text, and chunk-file paths are one-to-one for the 98 extracted papers. SQLite integrity checks pass. The observed coverage is therefore 98/134 papers (73.13%), with the remaining 26.87% unavailable to full-text retrieval. There is no OCR fallback, and section inference leaves 32.94% of chunks unknown.

10.2 Retrieval and answer traceability (RQ2)

Keyword FTS contains all 756 chunks. By contrast, the active hashing-vector JSON index contains 25 chunks from three papers (3.31% of chunks), while 300 SQLite rows retain an indexed-hashing status. Figure 10.1 visualizes this discrepancy.

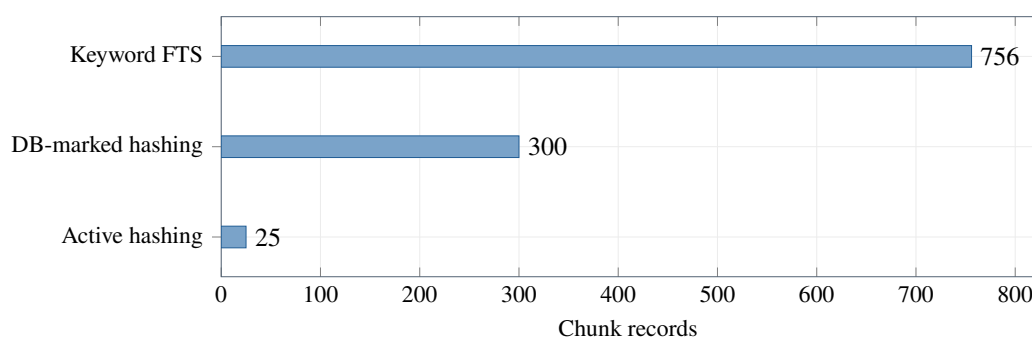


Figure 10.1: Keyword and hashing-index coverage. Database-marked hashing status overstates the 25-record active index.

The cause is reproducible in code: a bounded demo rebuild overwrites the JSON vector store but does not clear the status of omitted chunks. The API’s active semantic count reads the

JSON and correctly reports 25; database status alone is not trustworthy coverage evidence.

Representative disposable probes returned HTTP 200 for keyword, hashing-vector, and hybrid search and for three offline Ask requests. Returned answers included paper/chunk/page citations, retrieved chunks, provider/model metadata, warnings, and review state. All 27 persisted historical answers are structurally grounded and require review; 16 use the offline provider, ten record a qwen3 model, and one records qwen2.5. No human faithfulness, usefulness, or citation-correctness scores exist.

A broad probe about agriculture or AI began with relevant agriculture/machine-learning evidence but then included unrelated general AI/RAG material. This is direct failure evidence: structural grounding can coexist with a poorly bounded multi-intent answer.

10.3 Recommendation traceability (RQ3)

Seven persisted runs contain 29 recommendation items and 67 citations. Gap labels comprise three explicit, 16 inferred, and ten not found. Every run is partial and needs review. A disposable request returned three ranked recommendations with scores 0.5176, 0.5169, and 0.5002; only the first found an explicit gap in retrieved text. The implemented structures visibly separate source evidence and gap status from generated extension content (Fig. 8.6). No result demonstrates relevance, novelty, educational benefit, or feasibility.

10.4 Discovery, artefacts, and review (RQ4)

The explorer contains 39 fixed topics, 767 paper-topic links, and 1,105 author-topic links; 132 of 134 papers have a topic link. It also exposes unresolved identities and the “Click to View” parsing artefact. Fourteen persisted generated artefacts cover five papers, including five text-only podcast scripts; all require review. The database contains 182 items in the review queue and zero review events. Thus, inspectability and review mechanisms are implemented, while corrected data and completed human governance are not.

10.5 Reproducibility and verification (RQ5)

Backend tests executed against an isolated SQLite database: 71 passed with five PyMuPDF/SWIG deprecation warnings in 8.01 seconds. The test suite is not entirely filesystem-hermetic because one artefact test writes to the default generated-data path. A clean frontend install/build completed with 72 packages, zero reported vulnerabilities, and 1,796 transformed modules; produced asset sizes are documented in the evidence log.

Browser inspection exercised all eight main views without console or request errors. Final document-build and visual-QA results are recorded in thesis/PROJECT_EVIDENCE.md.

Table 10.1: Current evaluation results and what they do not establish.

Evaluation	Observed state	Prohibited inference
Retrieval	not run; gold paper IDs absent	no Recall@3/5 or MRR claim
QA	not run; answer points/gold citations absent	no faithfulness or usefulness claim
Extension	one explicit placeholder case, citation coverage 1.0	no relevance or feasibility claim
Artefact	one explicit placeholder case, citation coverage 1.0	no correctness or readability claim
Human review	templates and mechanism; zero events	no reviewer agreement or quality improvement claim
Ollama benchmark	absent; service unavailable during audit	no local-model comparison or latency claim

The primary result is therefore a working and auditable prototype with quantified coverage and identifiable failure modes, not validated retrieval or recommendation effectiveness.

Discussion

11.1 From archive to inspectable advice

The artefact advances beyond a publication list in a concrete sense: 98 papers are represented as page-aware evidence, retrieval outputs expose chunk/page provenance, recommendations separate retrieved gaps from generated proposals, and review status follows generated items. This integration makes the reasoning path more inspectable than an abstract-only or answer-only interface.

That same evidence reveals why the system should not yet be presented as a validated advisor. A user can see a citation but still receive an incomplete or topically diffuse answer. A student can see an attractive extension plan whose novelty and feasibility have never been reviewed. An industry user can see an author-topic link produced from a malformed identity. Traceability reduces the cost of checking; it does not perform the check.

11.2 RQ1: coverage is useful but incomplete

The ingestion pipeline demonstrates reproducible full-text processing for 73.13% of the catalogue. One-to-one derived files and successful database integrity checks support a bounded conclusion about internal consistency. Missing PDFs, absent OCR, heuristic sentence boundaries, and 32.94% unknown sections limit any stronger claim about corpus completeness or extraction quality. The platform should expose both paper-level availability and chunk-level diagnostics to users and evaluators.

11.3 RQ2: provenance survives, semantic coverage does not

Paper, page, chunk, provider, warning, and review fields survive the retrieval-to-display path. This is the strongest implementation result. However, the active hashing index covers only

three papers and its algorithm is lexical feature hashing, not learned semantic representation. Hybrid retrieval can still return results via FTS, but UI language and database status can overstate semantic coverage. A single transactional index manifest or rebuild procedure should make vector state authoritative.

The broad-intent failure also validates the project preference for representative questions rather than tests alone. Automated tests establish contractual behaviour; they cannot establish that a composed answer “makes sense” across heterogeneous evidence.

11.4 RQ3: recommendation structure is safer than recommendation quality

The recommendation schema is a useful design contribution because it records source evidence, explicit/inferred/missing gap status, generated scope, skills, risk, data assumptions, and evaluation suggestions separately. All observed runs being partial is arguably preferable to false certainty. Yet the weights and templates are design choices, not learned or empirically calibrated decisions. Supervisor/student review is necessary before any recommendation is used as thesis advice.

11.5 RQ4: reviewability is necessary but not sufficient

The normalized topic links, author-topic aggregation, review queue, and field-diff events support correction in principle. Zero review events show that the governance loop is open. Moreover, unauthenticated actions and a fixed reviewer name are unsuitable for public operation. Data cleaning should precede expertise claims, and review evidence should be versioned alongside generated outputs.

11.6 RQ5: reproducibility is stronger than evaluation maturity

The repository can be rebuilt, tested, probed, inspected, and compiled from documented commands. Hashes identify the data snapshot, while a disposable probe avoids modifying the source database. This supports reproducibility of the reported implementation state. It does not provide a stable experimental benchmark because gold labels, model benchmark outputs, hardware timings, repetitions, and reviewer judgments are absent.

11.7 Implications for research-intelligence systems

Three broader lessons follow. First, product labels should match algorithms: a hashing vector should not be silently equated with a learned semantic encoder. Second, the system of record for an index must include corpus and model identity, coverage, and status invalidation. Third, evaluation state should be a first-class interface output. The dashboard's honest not run states are more scientifically useful than a fabricated score.

Limitations, Ethics and Threats to Validity

12.1 Technical limitations

The corpus is incomplete, OCR is absent, section and sentence detection are heuristic, and metadata lacks structured abstracts, DOIs, and keywords. The active vector index is partial and its database statuses are stale. The only operational vector representation is lexical feature hashing. Hybrid and recommendation weights are not tuned. Topic classification uses a fixed taxonomy. Author identity resolution is incomplete. The OpenAI adapter is a stub, the local Ollama service was not reproducible during the audit, and text podcasts have no audio output.

The frontend lacks deep links, authentication, authorization, production deployment configuration, and a complete extraction-review surface. The dashboard can show empty top topics despite populated explorer data, and raw marker syntax can appear in the UI. These are not cosmetic details when the system is intended for public evidence communication.

12.2 Evaluation limitations

No valid retrieval or QA benchmark has been run. Placeholder citation-coverage cases cannot validate correctness. Persisted grounding labels test structure and low lexical overlap only. No human review events, reviewer scores, agreement measures, user study, ablation, latency benchmark, hardware record, statistical test, or external baseline comparison exists. The thesis is consequently unable to establish retrieval effectiveness, answer faithfulness, topic quality, recommendation usefulness, user satisfaction, or generalisability.

12.3 Ethical and responsible-AI considerations

Potential harms include misattributing research expertise, recommending infeasible or non-novel thesis work, exposing copyrighted text beyond justified quotation, retaining personal student profiles, and presenting generated interpretations as researcher-endorsed claims. Language-model systems also carry broader misinformation and interaction risks [30]. The interface’s generated notices, citations, gap labels, review states, and warnings are risk controls, but they require functioning governance.

Before public deployment the project needs:

- confirmation of document-processing and snippet-display rights;
- a privacy notice, lawful basis, retention policy, deletion path, and access controls for student inputs;
- verified author identity and a correction/appeal process;
- authenticated, attributable review with versioned approvals;
- accessibility, security, and abuse testing; and
- an institutional AI-assistance disclosure consistent with university policy.

AUTHOR INPUT REQUIRED: state ethics approval or exemption, document permissions, participant consent arrangements, data-retention decisions, and the required AI-use declaration.

12.4 Threats to internal validity

The audit reflects one working-tree snapshot that includes seven pre-existing uncommitted application/test modifications. Temporary database isolation reduces mutation risk, but one test writes an ignored generated artefact. Runtime probes use representative questions selected for diagnostic breadth rather than a preregistered sample. Manual interface inspection can miss intermittent or device-specific faults.

12.5 Threats to construct validity

The code’s Recall@k is a binary hit measure rather than set recall. Grounded denotes structural citation checks, not entailment. Topic score denotes lexical-rule evidence, not topic truth. Recommendation score denotes rule fit, not usefulness or feasibility. Treating these names as their stronger everyday meanings would invalidate conclusions.

12.6 Threats to external validity

The system is tailored to one laboratory archive and a small local corpus. Archive markup, publication genres, and metadata quality may differ elsewhere. Rule-based topics and query expansions are domain-specific. Results cannot be generalized to other institutions, corpora, languages, or production workloads without replication.

12.7 Threats to reproducibility

The ignored database and derived data are not wholly represented by Git history. A local model tag does not guarantee identical model bytes. External archive content can change. The thesis mitigates these issues with hashes, generated snapshots, explicit Git state, local build scripts, and separation of reproduced from historical evidence; a release should additionally archive the permitted corpus, model digest, dependency lockfiles, and environment image.

Conclusion and Future Work

This thesis examined how a laboratory publication archive can become a source-traceable research-intelligence workflow. The implemented prototype converts 98 of 134 records into 756 page-aware chunks, offers searchable and citation-returning interfaces, separates evidence from thesis-extension suggestions, links papers to topics and author strings, generates reviewable artefacts, and exposes evaluation state. Its repository can be tested, probed, built, and audited using the supplied evidence package.

The bounded answers to the research questions are as follows. RQ1: the pipeline provides internally consistent extraction for 73.13% of the snapshot, with missing PDFs, absent OCR, and incomplete section recognition. RQ2: retrieval and Ask TTLAB preserve paper/chunk/page provenance, but the active hashing index covers only 3.31% of chunks and structural grounding does not establish correctness. RQ3: recommendation contracts distinguish source gap status from system suggestions, but no relevance or feasibility study supports the scores. RQ4: topics, authors, outputs, and review queues are inspectable, while identity defects, absent authentication, and zero review events prevent governance claims. RQ5: the audited implementation is reproducible from documented artefacts, but empirical effectiveness remains unevaluated.

The principal conclusion is not that the platform is already an effective research advisor. It is that a demonstrable, evidence-aware prototype exists and its own data makes the next scientific work clear.

13.1 Prioritized future work

1. **Freeze and repair evidence state.** Rebuild the full vector index, clear stale statuses transactionally, record corpus/index/model manifests, and make the API expose coverage by mode.

2. **Improve corpus quality.** Resolve author identities, remove parsing artefacts, populate verified metadata, acquire permitted missing PDFs, add OCR with confidence diagnostics, and validate section extraction.
3. **Execute a gold retrieval/QA evaluation.** Establish reviewer-approved cases, compare modes and ablations, label claim–citation entailment, and report uncertainty and errors.
4. **Evaluate recommendations with stakeholders.** Separate novelty, feasibility, source fidelity, usefulness, risk calibration, and evaluation-plan quality; compare against a retrieval-only baseline.
5. **Complete governance.** Add authenticated review, reviewer identity, versioned approvals, change provenance, correction requests, retention/deletion controls, and deployment security.
6. **Evaluate learned alternatives.** Compare the current lexical hashing baseline with a reproducibly versioned scholarly or sentence encoder, rather than assuming semantic improvement [12, 14, 15].
7. **Improve public communication.** Repair deep links and dashboard inconsistencies, hide raw markers, expose extraction review, and conduct accessibility and usability testing.

13.2 Final author decisions

Student ID, supervisor, submission date, official laboratory naming, methodology designation, ethics status, corpus authorization, authorship/AI disclosure, and the proposed evaluation protocol remain **AUTHOR INPUT REQUIRED**. They are consolidated in `AUTHOR_REVIEW.md` rather than silently completed.

Research-Question Traceability Matrix

Table A.1: Connection from each research question to method, evidence, observed result, and conclusion.

RQ	Method	Evidence	Result	Conclusion
1	corpus, schema, and file audit	seed and PDF hashes; database and file manifests; extraction and chunk records	134 records; 98 extracted; 698 pages; 756 chunks; 35 missing; 1 failed; 32.94% unknown section	page-aware ingestion works for 73.13% of the snapshot; completeness and extraction correctness are not established
2	code inspection; active-index comparison; disposable search/Ask probes	FTS/index files; status fields; retrieval, answer, and verifier code; response JSON	FTS 756; active hashing 25; DB-marked hashing 300; responses preserve IDs, pages, and status; broad-intent noise observed	traceability is implemented; semantic coverage, answer correctness, and retrieval effectiveness are not established
3	schema, weight, and verifier inspection; persisted and disposable outputs	recommendation code and records	7 runs, 29 items, 67 citations; 3 explicit, 16 inferred, 10 missing gaps; all partial and unreviewed	evidence and suggestion separation exists; usefulness, novelty, and feasibility require human evaluation

RQ	Method	Evidence	Result	Conclusion
4	data and interface inspection	topics/links, author records, artefacts, review queue/events, screenshots	39 topics; identity defects; 14 artefacts; 182 queue items; zero events	inspectability exists; topic/expertise validity and governance do not
5	isolated tests, build, probes, hashes, PDF render QA	command results and generated evidence files	71 backend tests pass; frontend builds; API/UI surfaces run; documents build from source	implementation snapshot is reproducible; empirical experiments remain to be executed

Reproducibility Guide

Run commands from the repository root on Linux/WSL. The exact snapshot is Git HEAD 713ea5c8e321200eb5a17655481e091cab13ccb4 plus the seven modified application/test files enumerated in thesis/PROJECT_EVIDENCE.md.

B.1 Evidence snapshots

```
PYTHONPATH=backend .venv/bin/python thesis/scripts/collect_evidence.py  
PYTHONPATH=backend .venv/bin/python thesis/scripts/runtime_probe.py
```

The second command copies the SQLite database to a temporary location before requests. It should not add answer or recommendation rows to the source database.

B.2 Application verification

```
rm -f /tmp/advisor-thesis-pytest.db  
TTLAB_DATABASE_URL=sqlite:///tmp/advisor-thesis-pytest.db \  
  PYTHONPATH=backend .venv/bin/python -m pytest  
  
cd frontend  
npm ci  
npm run build
```

For local interactive operation, use the repository's scripts/run_everything.sh and inspect its options before execution. Local Ollama is optional; provider failure falls back to offline extraction.

B.3 Document builds

```
make thesis # build/thesis.pdf
make paper # build/ieee-paper.pdf
make all # both documents
```

The Makefile locates `tectonic` on `PATH` or under `$HOME/.local/bin`. Figure sources are TikZ/PGFPlots files under `thesis/figures/src`; plot inputs are generated CSV files.

B.4 Snapshot identifiers

Table B.1: Key SHA-256 snapshot identifiers.

Artefact	SHA-256
Seed JSON	d542403eee240fdae831af31f105ab1cc39f36a4b90eaa19f07b84f211b27b16
Runtime SQLite	d9474b25b224306c6cf8d5a3769e1483acf512e94b0c077040503c9c1693a9e2
Sorted PDF manifest	ac9ca99d2ee9c13a07442843b48712e51bef0effbf86f175682830f8a9ef53658
Active hashing index	8cb6843a...0511 (full value in generated evidence JSON)

B.5 Screenshots

The eight PNGs under `thesis/figures/screenshots` were captured from a disposable-copy browser audit on ports 8011/5181. They are evidence for visible surfaces and failures, not pixel-level regression baselines. Reproduction requires starting the audited backend copy and frontend and capturing the named views at a consistent viewport.

Schema and API Summary

C.1 Principal persisted contracts

Table C.1: Principal persisted entities and traceability fields.

Entity	Principal fields
Paper	identifier; title; author JSON; year/date/venue; abstract; source/PDF paths; DOI; keyword/topic JSON; ingestion, extraction, and review statuses; timestamps
Chunk	identifier; paper foreign key; page start/end; inferred section; text; word/token count; source hash; embedding status; timestamp
RAGAnswer	question; answer; retrieved chunk and paper JSON; citation JSON; provider/model; grounding status; unsupported-claim/warning JSON; human review fields; timestamp
Thesis	student request JSON; recommendation JSON; citation JSON;
Recommendation	grounding and review state; timestamp
PaperArtifact	paper foreign key; artefact type; generated text/JSON; citation and source-section JSON; provider/model; grounding and review state
Topic / PaperTopic / AuthorTopic	normalized topic; paper/author links; score; source and evidence JSON; review state
ReviewEvent	item type/ID; previous/new state; reviewer; notes; changed-field JSON; timestamp

C.2 Principal API surfaces

Table C.2: User-facing API groups and implemented purpose.

Group	Purpose
papers	statistics, list/detail, extracted status and paper artefacts
search	keyword, hashing-vector, or hybrid chunk retrieval with diagnostics
ask	retrieve, compose, cite, structurally verify, persist, and return answers
recommendations	profile-driven ranked paper/extension structures
topics/authors	topic list/detail, author list/detail, related papers, rebuild
admin	overview and review/update actions for metadata and generated items
evaluation	evaluation-state dashboard and available result files
local model	Ollama availability/model status used by the UI

Exact paths and schemas should be generated from the running FastAPI OpenAPI document for a release; this appendix intentionally avoids asserting a frozen public API version.

Proposed Review Instruments

These instruments are proposed and have not been administered. They require ethics and supervisor approval.

D.1 Answer and citation review

For each atomic answer claim, record: supported/partially supported/unsupported; correct paper; correct page/chunk; citation entails claim; important context omitted; conflicting evidence; and reviewer notes. At answer level, rate faithfulness and usefulness on a defined five-point rubric and record whether fallback occurred.

D.2 Recommendation review

Review source fidelity separately from the suggestion. Rate paper relevance, gap classification, novelty, feasibility within stated time, data accessibility, skill calibration, MVP clarity, risk calibration, and evaluation-plan appropriateness. Reviewers should flag where a system inference is phrased as if it appeared in the paper.

D.3 Topic and author review

For each sampled paper, mark applicable controlled topics, missing topics, incorrect topics, and ambiguous labels. For author records, first decide identity merges/splits and parsing errors, then review whether topic evidence supports a qualified expertise association.

D.4 Minimum provenance fields

Each judgment should retain case ID, corpus/index version, output ID, provider/model/prompt version, reviewer pseudonym, timestamp, decision, rationale, and adjudication state. **AUTHOR INPUT REQUIRED: approve instruments, scales, reviewer training, anonymity, retention, and adjudication procedure.**

Bibliography

- [1] T. Narine and P. Hosein, “Automating the collection, display, summarization and podcasting of academic research,” in *2025 IEEE 9th Forum on Research and Technologies for Society and Industry (RTSI)*. Tunis, Tunisia: IEEE, Aug. 2025, pp. 35–40. [Online]. Available: <https://doi.org/10.1109/RTSI64020.2025.11212505>
- [2] C. L. Giles, K. D. Bollacker, and S. Lawrence, “CiteSeer: An automatic citation indexing system,” in *Proceedings of the Third ACM Conference on Digital Libraries*. New York, NY, USA: ACM Press, 1998, pp. 89–98. [Online]. Available: <https://doi.org/10.1145/276675.276685>
- [3] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, “ArnetMiner: Extraction and mining of academic social networks,” in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: ACM, 2008, pp. 990–998. [Online]. Available: <https://doi.org/10.1145/1401890.1402008>
- [4] W. Ammar, D. Groeneveld, C. Bhagavatula, I. Beltagy, M. Crawford, D. Downey, J. Dunkelberger, A. Elgohary, S. Feldman, V. Ha, R. Kinney, S. Kohlmeier, K. Lo, T. Murray, H.-H. Ooi, M. Peters, J. Power, S. Skjonsberg, L. L. Wang, C. Wilhelm, Z. Yuan, M. van Zuylen, and O. Etzioni, “Construction of the literature graph in Semantic Scholar,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 3 (Industry Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 84–91. [Online]. Available: <https://aclanthology.org/N18-3011/>
- [5] J. Priem, H. Piwowar, and R. Orr, “OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts,” *arXiv preprint arXiv:2205.01833*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.01833>
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation

- for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html>
- [7] J. Beel, B. Gipp, S. Langer, and C. Breiter, “Research-paper recommender systems: A literature survey,” *International Journal on Digital Libraries*, vol. 17, no. 4, pp. 305–338, 2016, published online in 2015; assigned to the 2016 journal volume. [Online]. Available: <https://doi.org/10.1007/s00799-015-0156-0>
- [8] K. Sugiyama and M.-Y. Kan, “Scholarly paper recommendation via user’s recent research interests,” in *Proceedings of the 10th Annual Joint Conference on Digital Libraries*. New York, NY, USA: ACM, 2010, pp. 29–38. [Online]. Available: <https://doi.org/10.1145/1816123.1816129>
- [9] K. Balog, L. Azzopardi, and M. de Rijke, “Formal models for expert finding in enterprise corpora,” in *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. New York, NY, USA: ACM, 2006, pp. 43–50. [Online]. Available: <https://doi.org/10.1145/1148170.1148181>
- [10] —, “A language modeling framework for expert finding,” *Information Processing & Management*, vol. 45, no. 1, pp. 1–19, 2009. [Online]. Available: <https://doi.org/10.1016/j.ipm.2008.06.003>
- [11] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 4, no. 1–2, pp. 1–174, 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [13] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Online: Association for Computational Linguistics, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [14] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. Weld, “SPECTER: Document-level representation learning using citation-informed transformers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online:

- Association for Computational Linguistics, 2020, pp. 2270–2282. [Online]. Available: <https://aclanthology.org/2020.acl-main.207/>
- [15] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1, 2021. [Online]. Available: <https://openreview.net/forum?id=wCu6T5xFjeJ>
- [16] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, “Feature hashing for large scale multitask learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning*. New York, NY, USA: ACM, 2009, pp. 1113–1120. [Online]. Available: <https://doi.org/10.1145/1553374.1553516>
- [17] D. M. Blei, A. Y. Ng, and M. I. Jordan, “Latent dirichlet allocation,” *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003. [Online]. Available: <https://www.jmlr.org/papers/v3/blei03a.html>
- [18] M. Grootendorst, “BERTopic: Neural topic modeling with a class-based TF-IDF procedure,” *arXiv preprint arXiv:2203.05794*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.05794>
- [19] C. Doogan and W. Buntine, “Topic model or topic twaddle? re-evaluating semantic interpretability measures,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, 2021, pp. 3824–3848. [Online]. Available: <https://aclanthology.org/2021.naacl-main.300/>
- [20] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Online: Association for Computational Linguistics, 2021, pp. 874–880. [Online]. Available: <https://aclanthology.org/2021.eacl-main.74/>
- [21] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “SELF-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=hSyW5go0v8>
- [22] S. Es, J. James, L. E. Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. St. Julians, Malta: Association for Computational Linguistics, 2024,

- pp. 150–158. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>
- [23] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “ARES: An automated evaluation framework for retrieval-augmented generation systems,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 338–354. [Online]. Available: <https://aclanthology.org/2024.naacl-long.20/>
- [24] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 12 076–12 100. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.741/>
- [25] A. R. Fabbri, W. Kryściński, B. McCann, C. Xiong, R. Socher, and D. Radev, “SummEval: Re-evaluating summarization evaluation,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 391–409, 2021. [Online]. Available: https://doi.org/10.1162/tacl_a_00373
- [26] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 2511–2522. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.153/>
- [27] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the people: The role of humans in interactive machine learning,” *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014. [Online]. Available: <https://doi.org/10.1609/aimag.v35i4.2513>
- [28] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. [Online]. Available: <https://doi.org/10.1145/3458723>
- [29] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” in *Proceedings of the Conference on Fairness, Accountability, and Transparency*. New York, NY, USA: ACM, 2019, pp. 220–229. [Online]. Available: <https://doi.org/10.1145/3287560.3287596>
- [30] L. Weidinger, J. Uesato, M. Rauh, C. Griffin, P.-S. Huang, J. Mellor, A. Glaese, M. Cheng, B. Balle, A. Kasirzadeh, C. Biles, S. Brown, Z. Kenton, W. Hawkins,

- T. Stepleton, A. Birhane, L. A. Hendricks, L. Rimell, W. Isaac, J. Haas, S. Legassick, G. Irving, and I. Gabriel, “Taxonomy of risks posed by language models,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. New York, NY, USA: ACM, 2022, pp. 214–229. [Online]. Available: <https://doi.org/10.1145/3531146.3533088>
- [31] E. Tabassi, “Artificial intelligence risk management framework (AI RMF 1.0),” National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep. NIST AI 100-1, Jan. 2023. [Online]. Available: <https://doi.org/10.6028/NIST.AI.100-1>